

Memory Matching Game

Overall Description

This project is a Python memory matching game. The player sees a 4 by 4 board with 16 hidden cards. There are 8 matching pairs, using the letters A through H. The goal is to find every matching pair in the fewest number of moves.

The game has two playable versions:

- ``game.py``: a Pygame window with a menu, classic card-clicking mode, and input mode.
- ``cli_game.py``: a terminal version that can be played with typed commands.

Players

This is a human-computer game.

- Human player: chooses cards and tries to remember where values are.
- Computer: shuffles the board, checks matches, tracks moves, and can make auto-solver moves when the player presses ``S``.

Rules

1. The board starts with 16 face-down cards.
2. Each card has one hidden letter from A to H.
3. Every letter appears exactly twice.
4. The player selects two different unmatched cards per turn.
5. If the two cards have the same letter, they stay matched.
6. If the two cards do not match, they are hidden again.
7. One move is counted after the player selects a second card.
8. The game ends when all 8 pairs are matched.
9. The player can reset the game at any time.

Controls

Pygame Version

Run: ***python3 game.py***

Menu options:

- `Classic Mode`: click cards with the mouse.
- `Input Mode`: type card numbers from 1 to 16.
- `Quit`: close the game.

Keyboard shortcuts:

- `S`: auto-solve or auto-pick a turn.
- `R`: reset the game.
- `ESC`: return to the menu.

Terminal Version

Run: ***python3 cli_game.py***

Commands:

- `1` through `16`: select a card.
- `s`: let the solver choose.
- `r`: reset.
- `q`: quit.

Game Flow

The flowchart will be created separately.

Planned flow:

1. Start game.
2. Shuffle 8 pairs into 16 card positions.
3. Show menu.
4. Player chooses a mode.
5. Display the board.

6. Player selects the first card.
7. Player selects the second card.
8. Check whether the two cards match.
9. If they match, mark both cards as matched.
10. If they do not match, hide both cards again.
11. Check whether all cards are matched.
12. If all cards are matched, show the win screen.
13. If not, continue the next turn.

Implementation Tasks

The project is split into these files:

- ``memory_logic.py``: board creation, reset state, matched cards, move tracking, and solver logic.
- ``memory_constants.py``: Pygame screen size, colors, fonts, and board layout values.
- ``memory_sound.py``: generated sound effects and safe sound playback.
- ``memory_ui.py``: menu, cards, terminal-style board, input panel, and win popup drawing.
- ``game.py``: main Pygame event loop and game state controller.
- ``cli_game.py``: terminal version of the same memory game.
- ``test_memory_logic.py``: automated tests for important logic rules.

TEST CASES

MANUAL TEST CASES

Test 1: Vanilla gameplay

Steps: Start `game.py`, choose Classic Mode, and match all pairs.

Expected Result: The game counts moves and shows the win popup.

Test 2: Reset game

Steps: Start a game, flip a few cards, then press R.

Expected Result: The board reshuffles and the move count returns to 0.

Test 3: Menu navigation

Steps: Enter a game mode, then press ESC.

Expected Result: The game returns to the main menu.

Test 4: Invalid input edge case

Steps: In Input Mode, type 0, 17, or a non-number.

Expected Result: The game rejects the input and asks for a valid card.

Test 5: Same card edge case

Steps: In Input Mode, select the same card twice.

Expected Result: The game rejects the second selection and asks for a different card.

Test 6: Already matched edge case

Steps: Match a pair, then try selecting one of those matched cards again.

Expected Result: The game does not allow the matched card to be selected.

Test 7: Auto-solver

Steps: Press S during a game.

Expected Result: The computer selects cards and the game continues correctly.

Test 8: End game

Steps: Match all 8 pairs.

Expected Result: The win message appears, and the Play Again/Menu options work.

AUTOMATED TESTS

Install dependencies:

```
python3 -m pip install -r requirements.txt
```

Run tests:

```
python3 -m unittest
```

The automated tests check that:

The board always has 16 cards.

The board always has exactly 8 pairs.

Reset creates a clean new game state.

The auto-solver can use a known pair.

The auto-solver stops when fewer than two unmatched cards remain.

The terminal log keeps only the latest visible lines.

Github Link: <https://github.com/dilipBadal/MemoryCards>